

Non-Functional Requirements - Sprint 3

GLOW - Temp Tracker

Top 3 Prioritized NFRs

Performance

Performance is prioritized because the app relies on its visualization of large datasets and high success rate temperature uploads. With delays or lags, the core functionality of the app will remove the experience and prompt users to switch to competing programs. These requirements include the following:

- All pages must load under 1500ms
- The map must support an upper bound of 500k points if using a traditional plotting implementation. Otherwise, we would use server-side rendering with raster tiling to avoid crashing the browser. This aspect is especially prioritized for the next iteration.
- All web routes should have no 500- errors and have a 200- status or equivalent more than 95% of the time
- All DB reads/writes should be successful, and have a 95% success rate

Security

We also ensure security because our app deals with thousands of data points. This includes reducing spam, protecting user data, and limiting features to appropriate accounts.

- Uploads (including both temperature and photo) are rate-limited and validated by admin accounts
- Bulk uploads should only be performed by registered admin accounts
- RLS policies on our Supabase restrict users to interacting/updating only their own data. Admins should have access to read/update any user's data accordingly.
- Middleware will block certain pages (ex: profile) from being accessed without login

Usability

Many other temperature plotting software of the Great Lakes exist. Many of these sites have outdated interfaces, however, and lack key features. In order to stand out from the competition, our app should be fully responsive and support a number of accessibility features.

- Fully responsive mobile and desktop views
- Themes, button sizes, layouts, and modern and accessible
- About page, badge descriptions, and software intuition are all written on the website. This emphasizes the app's community potential and how it's used.

Trade Offs

Limited real-time updates

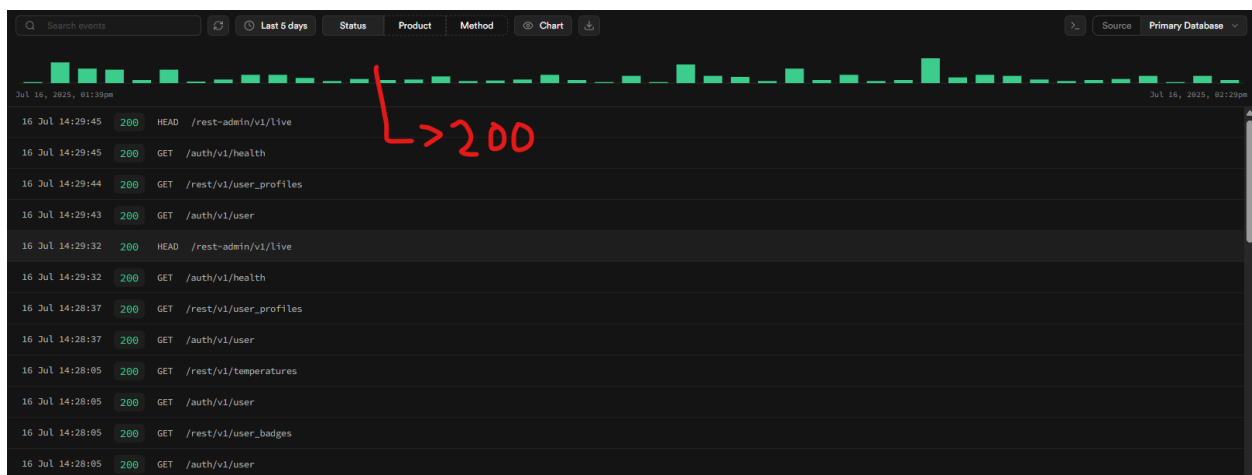
Due to performance issues and rendering challenges of the app, we will opt for on-demand and scheduled map updates. Using rasterized/tiled GeoJSON data to populate the map, we will lose the ability to continuously refresh the display for every point. This also means individual points might not be seen on the map, and a continuous display will be overlaid instead. This is done to limit client-side calculations and provide a lower latency, smoother experience for users. Using continuous renderings instead of discrete points may also increase the visual appeal of the map and increase usability while optimizing UI/UX.

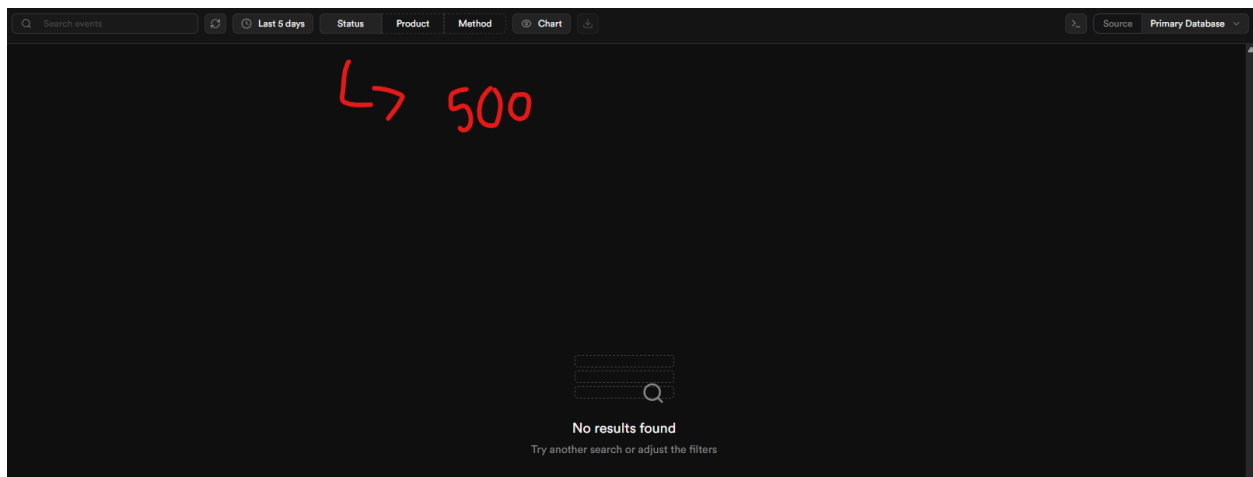
Rate-Limited Uploads

Genuine users uploading multiple points in succession will have to wait, due to rate-limited policies by our app. We need to ensure security and verify given data, meaning a large influx of spam data would need to be prevented. In terms of photo uploads, we need to limit photo uploads due to technical scaling requirements and a lack of large AWS cloud storage. Despite a shortened ecosystem, the app will be able to load images and approved points faster, increasing all aspects of the NFRs discussed above,

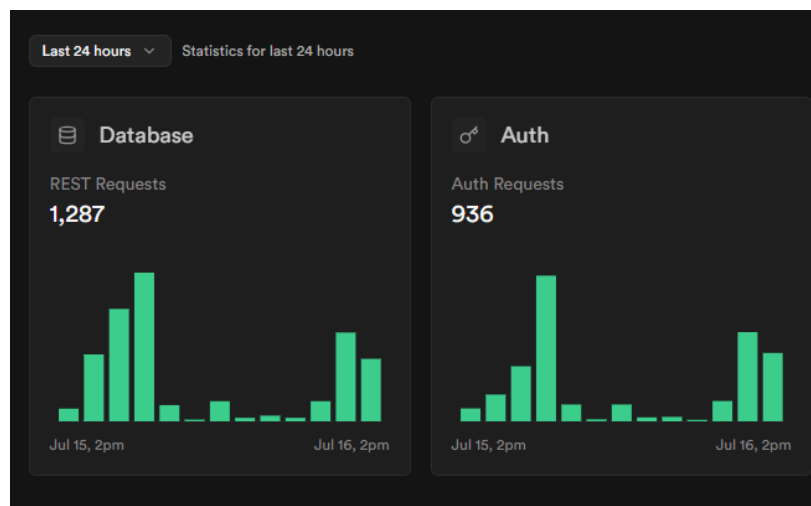
Performance Report

Using Supabase analytics, we can see the number of successful queries and login statuses of our app. Looking at the data, we can see our performance metrics align with our goals, as the DB queries have a >99% success rate. Looking at our Supabase Auth status, we can see we have zero 500- status codes.

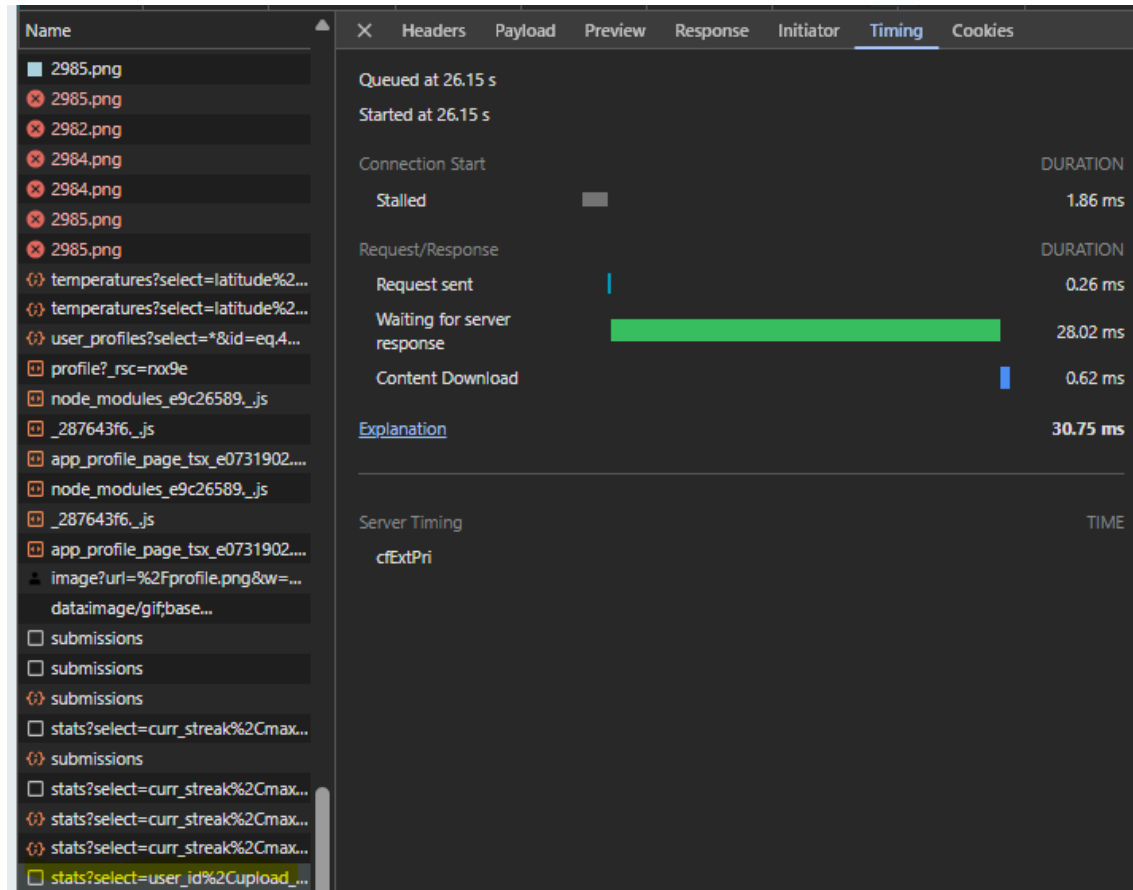




Additionally, when looking at metrics for the last 24 hours, we see we have over 1200 successful REST DB requests and over 900 successful logins. These metrics are repeated, regardless of the day selected for logs.



All network requests also have very minimal latency. As an example, when the user goes to the leaderboard page, the leaderboard defaults to showing the top 30 users by upload count. In the network requests, we are able to see this exact GET request, and after sending the `user_id` and `'upload_count'` parameters, we are able to retrieve the populated table. The full flow took less than 31ms, showing extremely low latency and great performance.



Security Measures

First off, authentication is handled by Subapase Auth, which uses JSON Web Tokens. Supabase also integrates with OAuth, allowing users to sign in with their Google accounts. These systems allow for automatic password hashing in an encrypted users table, which also securely stores metadata and login Tokens.

Authorization is enforced using Supabases' Row-Level Security (RLS) policies, shown below. Each table is appropriately limited to a specific number of users and functions. Additionally, middleware blocks access for unauthorized users from accessing pages such as /profile and /leaderboard.

schema: public		Filter tables and policies	Docs
🔗	badges	Disable RLS Create policy	
Note: Row Level Security is enabled, but no policies exist. No data will be selectable via Supabase APIs. No policies created yet			
🔗	stats	Disable RLS Create policy	
UPDATE	Allow streak updates for logged-in users Applied for: public role		1
SELECT	Enable read access for all users Applied for: public role		1
🔗	temperatures	Disable RLS Create policy	
INSERT	Enable insert for authenticated users only Applied for: authenticated role		1
SELECT	Enable read access for all admins Applied for: public role		1
SELECT	Enable read access for all users Applied for: public role		1
SELECT	Enable read access for user row Applied for: public role		1
UPDATE	Update Access Admins Applied for: public role		1
🔗	user_badges	Disable RLS Create policy	
Note: Row Level Security is enabled, but no policies exist. No data will be selectable via Supabase APIs. No policies created yet			
🔗	user_profiles	Disable RLS Create policy	
INSERT	Auth admin can insert user_profiles Applied for: supabase_auth_admin role		1
SELECT	authenticated user read Applied for: public role		1

Additionally, SQL Injection is prevented by both the frontend Supabase SDK and the ORM used in the Express backend. For POSTing data, we also have strict type constraints and validations, which the system tells the user. For example, in CSV uploads, explicit type checks and roles determine what kind of data is able to be uploaded, and the image below shows an admin-protected, strict verification with prompts telling the uploader how to proceed.

ADMIN ONLY

Bulk CSV Upload

Upload multiple temperature readings from a CSV file. This feature is restricted to authorized administrators.

×

submissions (2).csv ×

0.3 KB

×

Validation failed

Please check the console for details on the errors.

Required CSV headers:

temperature, temperatureUnit, date, time, longitude, latitude, notes

Submit CSV File

Scalability Considerations

Supabase's PostgreSQL instance uses automatic vertical scaling on paid plans to match our given data. Currently, the team uses a free plan, but given extended loads, this can be vertically scaled appropriately. For large rasterized image tiles, we use Supabase Buckets to store images, but for future availability can switch to AWS S3 buckets for additional cloud storage. AWS can be configured to scale for us, and flexible cloud storage would be enough to match our business needs.

For load balancing/caching strategies, static pages are automatically balanced depending on the deployment service (ex: Vercel). A stateless backend API allows horizontal scaling if needed, and separate functions (ex: uploading, maps) can be isolated to a microservice architecture in a very large environment. Given the estimated user base, this will probably be unlikely, and vertical scaling appropriately would be sufficient.

In terms of the map, data tiles/heatmaps for the previous 3-7 days will be cached in buckets as mentioned above, to prevent rerendering commonly accessed map points.